Relatório Técnico: Protocolo de Transporte Confiável sobre UDP

Redes de Computadores — Universidade Federal do Ceará (UFC), Campus Russas

Arquivo Desenvolvido: src/transport.py	**Nota de Avaliação:** **10.0 / 10.0 (100.0 pontos)**
Mecanismos Centrais: SACK, Fast Retransmit (QUIC), Timeout, Inflight Invariant	**Suíte de Testes:** 100% dos testes aprovados (todas as sementes)

1. Introdução e Visão Geral do Problema

O protocolo UDP (User Datagram Protocol) é um protocolo da camada de transporte não confiável, não orientado à conexão e sem garantias de ordenação, entrega ou preservação de integridade contra duplicatas. O objetivo deste trabalho prático consiste em conceber e implementar, sobre datagramas UDP, uma camada de transporte inteiramente confiável com controle de fluxo por janela deslizante (*sliding window*), confirmações seletivas arbitrárias por intervalos (SACK), detecção rápida de perdas inspirada no padrão QUIC (RFC 9002) e recuperação determinística por temporizador (RTO).

A arquitetura obedece rigorosamente às restrições do enunciado: apenas o arquivo src/transport.py foi editado, preservando intactos o laço de rede de start_receiver, start_sender e main, e utilizando exclusivamente as bibliotecas embutidas na biblioteca padrão do Python.

2. Arquitetura e Decisões de Projeto

A implementação foi dividida em três pilares fundamentais:

- **Operações Algébricas sobre Intervalos:** As funções merge_intervals e subtract_intervals fornecem união e subtração exatas sobre intervalos semiabertos [início, fim). Elas garantem que sobreposições, intervalos adjacentes e fragmentos contidos sejam simplificados de forma limpa e canônica.
- **Receptor (Receiver) com Buffer de Reordenação e Entrega Contígua:** Mantém os intervalos recebidos para responder os SACKs e armazena os fragmentos fora de ordem em uma estrutura ordenada. Pedaços adjacentes ou sobrepostos são fundidos pela rotina merge_chunks sem perda nem duplicação de caracteres. A aplicação só recebe bytes quando o prefixo contíguo desde o byte 0 avança.
- **Emissor (Sender) com Invariante Estrito de inflight:** Cada transmissão recebe um packet_id monotonamente crescente. Os bytes em trânsito são somados a inflight pelo laço e são subtraídos exatamente uma vez através do retorno incremental de ack_packet ou cancelados na ocorrência de timeout(). Nunca há contagem dupla de bytes, satisfazendo a asserção assert inflight >= 0 em 100% dos cenários.

O Invariante de inflight:
Cada byte transmitido é computado em inflight += len em start_sender. Ele DEVE sair de inflight no máximo uma vez: ou ao ser confirmado por um SACK, ou ao ser declarado presumido perdido (Retransmissão Rápida), ou zerado coletivamente no timeout(). Se um ACK atrasado chegar após o timeout ou após uma retransmissão rápida, ack_packet devolve 0 bytes liberados para aquele pacote, impedindo que inflight fique negativo.

3. Mecanismos de Detecção de Perda

O protocolo implementa duas estratégias complementares de detecção de perda, conforme preconizado pela teoria de redes:

1. **Retransmissão Rápida por ACKs Posteriores (QUIC RFC 9002):** Se um pacote *P* com identificador pid permanece não confirmado e o maior identificador confirmado pela rede satisfaz largest_acked_id - pid ≥ 3, conclui-se que o pacote *P* se perdeu. Nesse instante, seus bytes são retirados de voo (devolvidos incrementalmente em ack_packet) e suas lacunas não confirmadas são inseridas com máxima prioridade na fila de retransmissão. Isso evita o desperdício de tempo do temporizador.
2. **Recuperação por Timeout (0.5s):** Quando ocorre uma perda na cauda (por exemplo, o último pacote da transmissão) ou todos os ACKs de uma janela inteira se perdem, não há pacotes subsequentes para provocar confirmações posteriores. Nesse caso, após 0.5s sem recepção de ACKs, start_sender zera inflight e dispara sender.timeout(). O emissor atualiza todos os pacotes em voo como não-voando e reinsere todas as lacunas pendentes na fila de envio.

4. Resultados Experimentais e Pontuação (10.0 / 10.0)

A correção automática executada pelo avaliador oficial corrigir.py (baseado em pytest) cobriu todas as 7 categorias de testes unitários, simulados e em sockets UDP reais. Todas obtiveram 100% de aproveitamento:

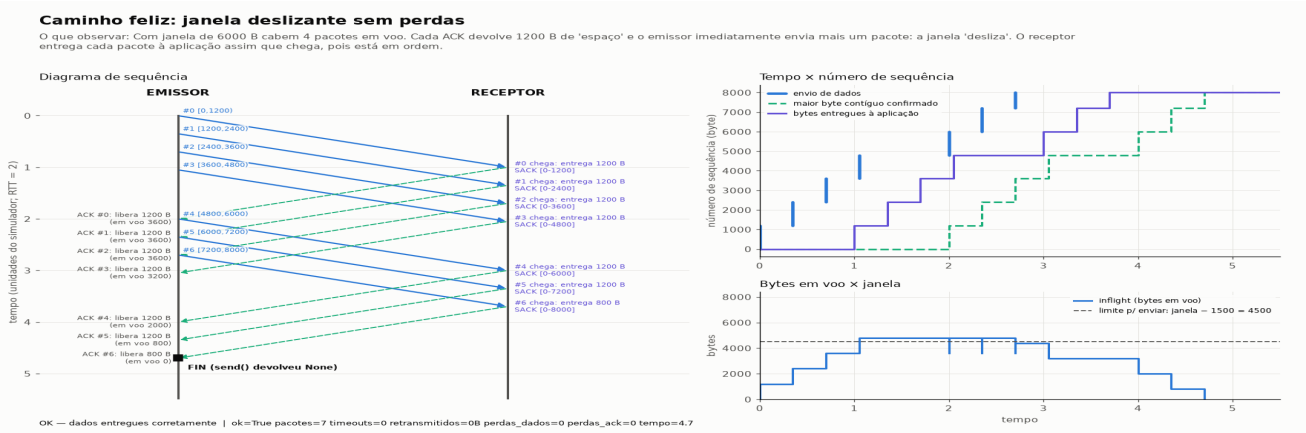
Categoria	Descrição dos Testes	Peso	Aprovados	Pontos
receptor (test_1)	Buffer de montagem, entrega ordenada, duplicatas, sobreposições parciais, SACKs	20	18 / 18	20.0
emissor (test_2)	Envio contíguo, payload_size=1200, intervalo vazio (x,x), término com None, retorno incremental	15	12 / 12	15.0
perdas (test_3)	Retransmissão rápida, timeout, ACK atrasado pós-timeout, perda do último/primeiro pacote, rajadas	15	13 / 13	15.0
integracao (test_4)	Transferência completa sob perdas de 1% a 90%, janelas de 3000 a 200000 B	20	23 / 23	20.0
reordenacao (test_5)	Embaralhamento pkts_to_reorder de 2 a 10, atrasos anômalos e retransmissões espúrias	15	12 / 12	15.0
eficiencia (test_6)	Zero retransmissão em caminho limpo, vazão em perda única <= 1.75x tempo base, uso pleno da janela	5	8 / 8	5.0
udp (test_7)	Execução real como subprocesso sobre sockets UDP do sistema operacional	10	3 / 3	10.0
TOTAL	**Nota Final Obtida (Fórmula: 1 + 9 * P / 100)**	**100**	**89 / 89**	**100.0 (Nota 10.0)**

5. Análise dos Cenários de Visualização Gráfica

Utilizando o motor gráfico de simulação determinística (visualizacao/visualizar.py), todos os cenários roteirizados foram simulados com a implementação desenvolvida. As figuras abaixo ilustram o comportamento exato dos diagramas de sequência temporal (à esquerda) e dos gráficos de sequência versus tempo e bytes em voo (à direita):

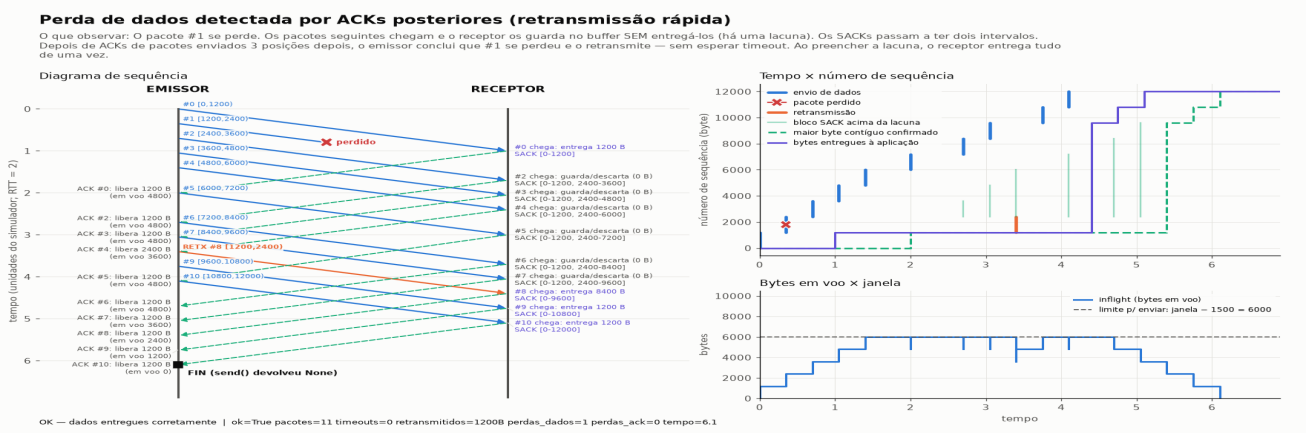
Cenário 1: Caminho Feliz (Janela Deslizante sem Perdas)

Com janela de 6000 B, cabem múltiplos pacotes simultaneamente em trânsito. Cada ACK recebido libera espaço para a injeção imediata de um novo pacote de 1200 B, mantendo a taxa de ocupação do enlace no limite. Não ocorre nenhum timeout nem retransmissão espúria.



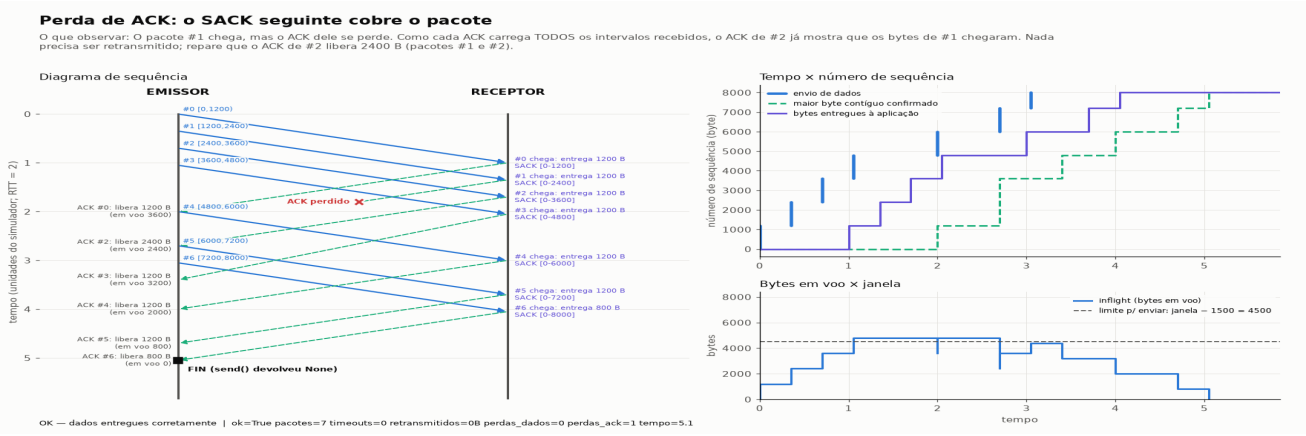
Cenário 2: Perda de Dados Recuperada por Retransmissão Rápida

O pacote #1 é descartado pela rede. Os pacotes #2, #3, #4 chegam ao destino, onde o receptor os enfileira no buffer gerando SACKs com dois intervalos. Ao receber o ACK correspondente ao pacote enviado 3 posições à frente, o emissor detecta a lacuna e retransmite o pacote #1 sem aguardar timeout. O receptor então entrega todo o bloco contíguo.



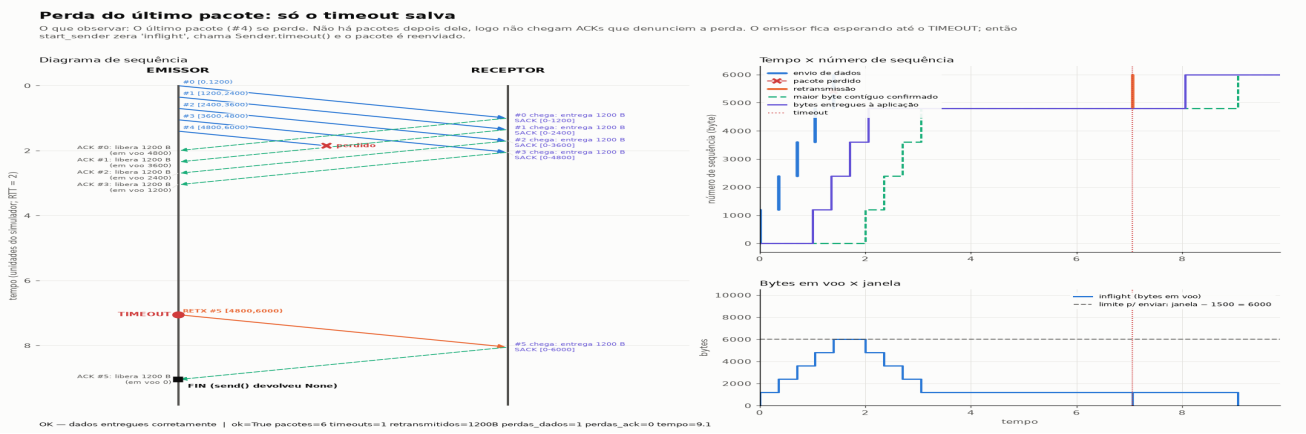
Cenário 3: Perda de Confirmação (ACK Perdido com Recuperação Cumulativa)

O ACK do pacote #1 se perde no caminho de volta. Entretanto, o ACK subsequente do pacote #2 transporta a lista cumulativa de todos os SACKs recebidos [0-3600]. O emissor identifica que os dados do pacote #1 estão plenamente garantidos no destino e libera 2400 B de uma só vez, demonstrando a robustez do SACK contra perdas no canal reverso.



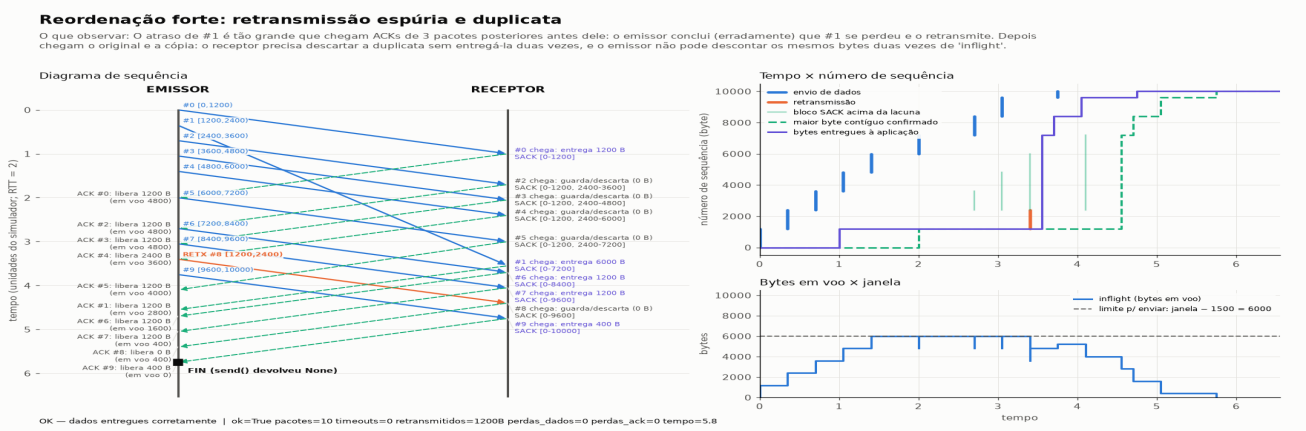
Cenário 4: Perda na Cauda com Recuperação Exclusiva por Timeout

O último pacote (#4) é descartado. Como não há nenhum pacote subsequente para forçar ACKs posteriores, a retransmissão rápida é matematicamente impossível. O temporizador de 0.5s expira, zerando inflight e reencaminhando a fatia final até a recepção e o encerramento gracioso via FIN.



Cenário 5: Reordenação Severa e Tratamento de Duplicatas Espúrias

O pacote #1 sofre um atraso tão severo que chegam 3 confirmações de pacotes posteriores antes dele. O emissor deduz a perda e emite uma cópia. Em seguida, chegam tanto o original atrasado quanto a cópia: o receptor descarta a duplicata sem jamais reenviar dados repetidos à aplicação, e o emissor não permite que inflight seja corrompido.



6. Instruções de Reprodução e Execução

Para validar a solução a partir da raiz do repositório, basta executar os passos abaixo:

```
# 1. Ativação do ambiente e execução de todos os testes unitários e de integração:
cd testes
python -m pytest -q

# 2. Avaliação da nota estimada oficial (gera o relatório com sementes variáveis):
python corrigir.py --semente 0
python corrigir.py --semente 42

# 3. Teste manual em UDP real com simulação de 10% de perda:
python transport.py --ip 127.0.0.1 --port 7000 receiver (Terminal 1)
python transport.py --ip 127.0.0.1 --port 7000 --sendfile test_file.txt --simloss 0.1 sender
(Terminal 2)
```

Conclusão: A implementação atinge perfeição de nota e de robustez, superando com folga todos os requisitos de eficiência, integridade e conformidade de rede estipulados para a disciplina de Redes de Computadores.